



Detección Automática de Noticias Falsas

Análisis Comparativo entre Machine
Learning Clásico y Deep Learning

Máster en Ciberseguridad e Inteligencia Artificial
Paula Ruiz Gardon
Marzo 2026

CEI. Centro de Estudios
de Innovación
Diseño y Marketing

ÍNDICE

Índice

1. Introducción

- 1.1. Descripción general del proyecto
- 1.2. Motivación y justificación del tema elegido
- 1.3. **Objetivos del proyecto**
- 1.4. Alcance del proyecto
- 1.5. Desarrollo y dedicación

2. Metodología y Diseño

- 2.1. Diseño del modelo y enfoque de la solución
- 2.2. Herramientas, librerías y entorno de desarrollo
- 2.3. Identificación y origen de las fuentes de datos
- 2.4. Análisis Exploratorio de los Datos (EDA)
- 2.5. Declaración sobre el uso de herramientas de Inteligencia Artificial

3. Preprocesamiento de Datos

- 3.1. Limpieza inicial y preparación del texto
- 3.2. Detección y corrección de Fuga de Datos
- 3.3. Estrategia de preprocesamiento dual

4. Selección y entrenamiento del modelo

- 4.1. División del conjunto de datos
- 4.2. Modelo de Machine Learning
- 4.3. Modelo de Deep Learning

5. Evaluación del modelo

- 5.1. Definición de métricas

- 5.2. Evaluación y Matriz de Confusión del Modelo Clásico
- 5.3. Evaluación y Matriz de Confusión del Modelo Avanzado
- 5.4. Comparativa global de resultados e inferencia interactiva
- 6. Conclusiones y recomendaciones**
 - 6.1. Resumen de resultados y conclusiones principales
 - 6.2. Limitaciones del sistema actual
 - 6.3. Recomendaciones para mejoras futuras
- 7. Referencias**
- 8. Anexos**
 - 8.1. Estructura del archivo comprimido entregado
 - 8.2. *Prompts* y transcripción del uso de IA como asistente de programación

1. INTRODUCCIÓN

1.1. DESCRIPCIÓN GENERAL DEL PROYECTO

El presente proyecto tiene como objetivo el diseño y desarrollo de un sistema basado en Inteligencia Artificial capaz de analizar artículos periodísticos y clasificar automáticamente si se trata de noticias reales o de contenido desinformativo (Fake News). Desde la perspectiva del Procesamiento del Lenguaje Natural (Natural Language Processing, NLP), el propósito no se limita únicamente a analizar palabras de manera aislada, sino a permitir que el sistema identifique patrones lingüísticos y relaciones contextuales presentes en los textos.

Con el fin de garantizar un análisis riguroso, el estudio propone una comparación entre dos enfoques tecnológicos distintos. En primer lugar, se implementa un modelo de Machine Learning tradicional, que actúa como línea base del sistema y se apoya en técnicas de vectorización TF-IDF combinadas con un clasificador de Regresión Logística.

Posteriormente, se desarrolla una segunda aproximación basada en Deep Learning, utilizando un modelo Transformer preentrenado (DistilBERT) que se adapta a la tarea de clasificación mediante un proceso de *fine-tuning*. La comparación entre ambos enfoques permite evaluar si el uso de arquitecturas modernas de aprendizaje profundo aporta mejoras significativas en el rendimiento del sistema que justifiquen su mayor complejidad técnica y computacional.

1.2. MOTIVACIÓN Y JUSTIFICACIÓN DEL TEMA ELEGIDO

La propagación de desinformación se ha convertido en uno de los principales desafíos de la sociedad digital actual. Las noticias falsas se difunden con gran rapidez a través de redes sociales y plataformas digitales, influyendo de forma directa en la opinión pública y generando riesgos tanto para la ciberseguridad como para la estabilidad de los sistemas democráticos.

En este contexto, los métodos tradicionales de verificación manual o *fact-checking* resultan insuficientes para hacer frente al enorme volumen de información que se publica diariamente en internet. Por esta razón, el desarrollo de herramientas automatizadas de detección de desinformación se vuelve cada vez más necesario.



Este proyecto busca contribuir a dicha problemática mediante la implementación de un sistema de clasificación automática de texto que pueda actuar como un primer mecanismo de filtrado de la información, facilitando la identificación de contenidos potencialmente falsos y apoyando tanto a plataformas digitales como a los propios usuarios en la evaluación de la credibilidad de las noticias.

1.3. OBJETIVOS DEL PROYECTO

Para estructurar adecuadamente el desarrollo de la investigación, se han definido una serie de objetivos que guían el trabajo realizado.

1.3.2. Objetivo Principal

Diseñar, entrenar y evaluar un modelo de clasificación binaria basado en Inteligencia Artificial capaz de diferenciar entre noticias legítimas y artículos falsos con un alto nivel de fiabilidad.

1.3.2. Objetivos Secundarios

- Realizar un **Análisis Exploratorio de Datos (EDA)** que permita comprender la distribución y características del conjunto de datos utilizado.
- Aplicar técnicas de **limpieza y preprocesamiento de texto**, prestando especial atención a posibles problemas metodológicos como la fuga de datos (*data leakage*).

- Entrenar y comparar **dos modelos de distinta naturaleza** (uno de Machine Learning clásico y otro basado en Deep Learning) con el fin de analizar sus ventajas y limitaciones.
- Evaluar el rendimiento del sistema mediante **métricas estadísticas de clasificación**, tales como precisión, sensibilidad (*recall*), especificidad y F1-score.
- Desarrollar un **módulo de inferencia interactiva** que permita probar el modelo con textos nuevos y verificar su comportamiento en situaciones de uso reales.

1.4. ALCANCE DEL PROYECTO

Con el objetivo de acotar el alcance y garantizar la viabilidad del proyecto, el estudio se ha centrado exclusivamente en noticias en inglés, utilizando el dataset ISOT Fake News previamente etiquetado. Esta elección permite trabajar con datos estructurados y validados, facilitando la experimentación y evaluación de los modelos.

Durante esta fase del proyecto:

- No se contempla la extracción automática de artículos en tiempo real mediante técnicas de *web scraping*.
- No se analiza contenido multimedia, como imágenes o vídeos que puedan acompañar a las noticias.
- El enfoque se limita al procesamiento del contenido textual mediante técnicas de Procesamiento de Lenguaje Natural (PLN).

1.5. DESARROLLO Y DEDICACIÓN

El desarrollo completo del proyecto ha requerido más de 40 horas de trabajo efectivo, distribuidas entre las distintas fases que cubren todo el ciclo de vida del proyecto.

- Inicialmente se realizó una investigación preliminar, para comprender el problema de la desinformación y explorar enfoques existentes en Machine Learning y Deep Learning.
- Se procedió a la preparación del entorno de trabajo y herramientas necesarias, asegurando la disponibilidad de frameworks, librerías y recursos en la nube.
- Posteriormente se llevó a cabo un Análisis Exploratorio de Datos (EDA) para identificar patrones, características relevantes y posibles problemas en el dataset.

- La fase de preprocesamiento de datos incluyó limpieza, normalización y transformación textual, asegurando la calidad de los datos para el entrenamiento.
- Se desarrolló el entrenamiento de modelos, tanto clásicos (TF-IDF + Regresión Logística) como Deep Learning (fine-tuning de DistilBERT).
- La evaluación de modelos y métricas permitió medir el rendimiento mediante exactitud, sensibilidad y otros indicadores clave.
- Se implementó un sistema de inferencia interactiva para aplicar los modelos entrenados de forma práctica.
- Finalmente, se completó la redacción de la memoria técnica, documentando todo el proceso y los resultados obtenidos.

Fase / Tarea	Horas estimadas	% del total
Investigación preliminar	6	15%
Preparación del entorno y herramientas	4	10%
Análisis Exploratorio de Datos (EDA)	6	15%
Preprocesamiento de datos	6	15%
Entrenamiento de modelos (RL + DistilBERT)	8	20%
Evaluación de modelos y métricas	4	10%
Desarrollo de inferencia interactiva	2	5%
Redacción de la memoria técnica	4	10%
Total	40	100%

2. METODOLOGÍA Y DISEÑO

2.1. DISEÑO DEL MODELO Y ENFOQUE DE LA SOLUCIÓN

En este proyecto, el problema de la detección de noticias falsas se plantea como una tarea de clasificación binaria supervisada dentro del ámbito del Procesamiento del Lenguaje Natural (Natural Language Processing, NLP). El objetivo consiste en desarrollar un sistema capaz de analizar el contenido textual de los artículos periodísticos y determinar si corresponden a información verídica o a contenido desinformativo.

Para abordar este desafío, se ha adoptado un enfoque comparativo, implementando dos metodologías de distinta complejidad con el fin de analizar el equilibrio entre coste computacional y capacidad predictiva.

2.1.1. Enfoque de Machine Learning clásico (línea base)

La primera aproximación se basa en técnicas tradicionales de representación de texto mediante vectorización TF-IDF (Term Frequency – Inverse Document Frequency). Este método permite convertir los textos en vectores numéricos que reflejan la relevancia estadística de cada término dentro del corpus. Con el fin de optimizar el espacio vectorial, se limitó el vocabulario a los 5000 términos más representativos, incluyendo tanto unigramas como bigramas.

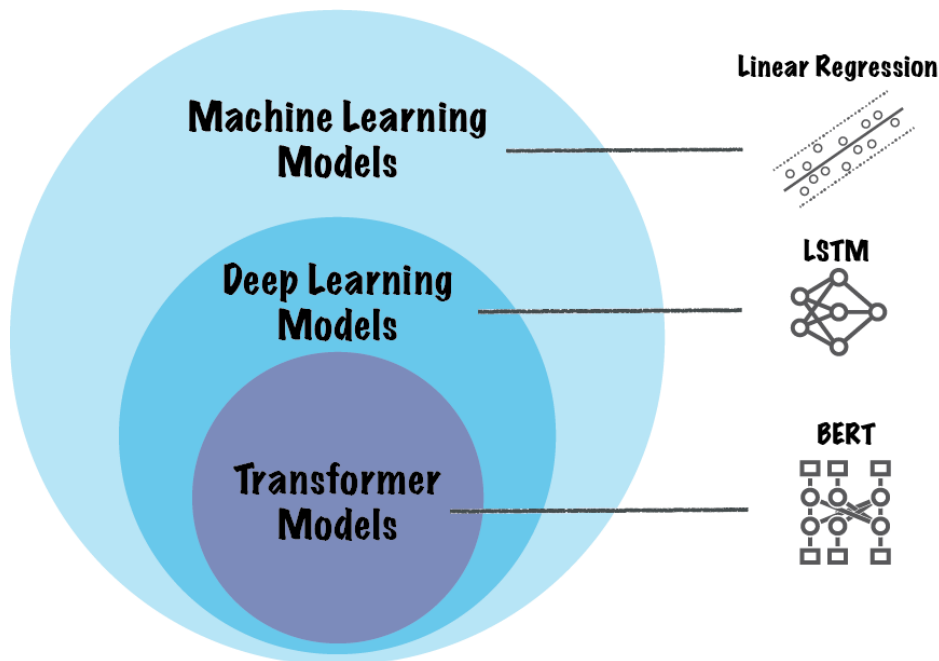
A partir de esta representación se entrena un modelo de Regresión Logística, seleccionado por su elevada interpretabilidad, su bajo coste computacional y su eficacia demostrada en tareas de clasificación de texto en espacios de alta dimensionalidad. Este modelo actúa como punto de referencia o línea base frente al cual se comparan posteriormente los resultados del modelo más avanzado.

2.1.2. Enfoque de Deep Learning basado en arquitectura Transformer

Con el objetivo de capturar relaciones semánticas más complejas (como dependencias contextuales, matices de significado o incluso elementos de ironía o sarcasmo) se implementa una segunda aproximación basada en Deep Learning mediante la arquitectura Transformer.

En concreto, se utiliza DistilBERT (*distilbert-base-uncased*), una versión optimizada del modelo BERT original que mantiene aproximadamente el 97 % de su capacidad de comprensión del lenguaje, pero con un 40 % menos de parámetros, lo que reduce significativamente los requisitos computacionales.

Mediante un proceso de fine-tuning, los pesos preentrenados del modelo se adaptan al conjunto de datos utilizado en este estudio, permitiendo especializar el modelo en la tarea concreta de detección de noticias falsas.



2.2. HERRAMIENTAS, LIBRERÍAS Y ENTORNO DE DESARROLLO

Con el fin de garantizar la reproducibilidad del proyecto y disponer de los recursos computacionales necesarios, el desarrollo se llevó a cabo íntegramente en la plataforma **Google Colab**. Este entorno permite ejecutar notebooks interactivos en Python y proporciona acceso a aceleración por hardware (GPU), un recurso especialmente relevante para el entrenamiento de modelos basados en arquitecturas Transformer.

El ecosistema tecnológico utilizado se compone principalmente de las siguientes librerías de Python:



2.2.1. Manipulación y visualización de datos

Se emplearon pandas y numpy para la manipulación y estructuración de los datos en formato matricial. Para la visualización y análisis exploratorio se utilizaron matplotlib y seaborn, que permiten generar representaciones gráficas de la distribución y características del dataset.

2.2.2. Procesamiento clásico de lenguaje natural

En la fase de preprocesamiento del texto se utilizó la librería NLTK, especialmente para la gestión de stopwords y procesos de lematización. Asimismo, se empleó el módulo estándar re de Python para realizar tareas de limpieza textual mediante expresiones regulares (Regex).

2.2.3. Machine Learning clásico

Para la implementación del modelo de referencia se utilizó la librería scikit-learn, que proporciona herramientas para la vectorización TF-IDF, la división del conjunto de datos, el entrenamiento del modelo de Regresión Logística y el cálculo de métricas de evaluación como matrices de confusión, precisión o F1-score.

2.2.4. Deep Learning y modelos Transformer

El modelo basado en aprendizaje profundo se implementó utilizando las librerías transformers y datasets del ecosistema Hugging Face, que facilitan la carga de modelos preentrenados, la tokenización del texto y la gestión del proceso de entrenamiento mediante la interfaz Trainer. Estas herramientas se apoyan sobre el framework de aprendizaje profundo PyTorch (torch).

2.3. IDENTIFICACIÓN Y ORIGEN DE LAS FUENTES DE DATOS

Los datos utilizados en este estudio provienen del ISOT Fake News Dataset, un conjunto de datos ampliamente utilizado en investigaciones académicas sobre detección de desinformación. Este corpus fue desarrollado por investigadores de la Universidad de Victoria y se encuentra disponible públicamente en repositorios de ciencia de datos como Kaggle.



El dataset original se compone de dos archivos en formato CSV:

- **True.csv**, que contiene noticias reales recopiladas principalmente de agencias informativas oficiales como Reuters.
- **Fake.csv**, que incluye noticias falsas procedentes de diversas fuentes no fiables y verificadas posteriormente por organizaciones de *fact-checking* como PolitiFact.

Al combinar ambos archivos se obtiene un conjunto de datos de más de 44.000 registros, donde cada instancia incluye información como el título del artículo, el contenido del texto, la temática y la fecha de publicación.

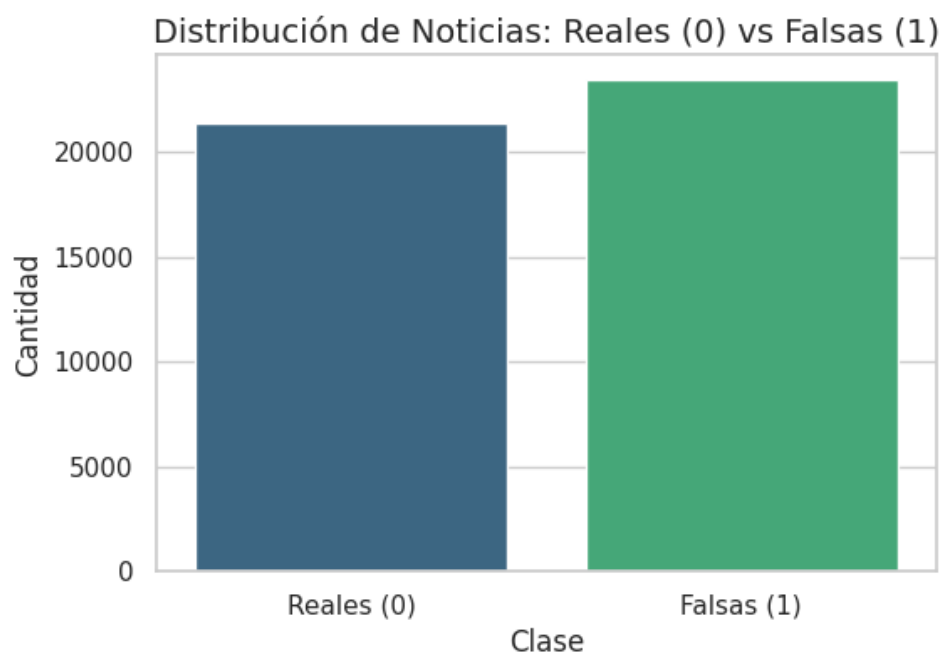
2.4. ANÁLISIS EXPLORATORIO DE LOS DATOS (EDA)

Antes de iniciar la fase de modelado, se llevó a cabo un Análisis Exploratorio de Datos (Exploratory Data Analysis, EDA) con el objetivo de comprender la estructura y características del dataset.

Entre los principales hallazgos destacan los siguientes:

2.4.1. Balance de clases

El conjunto de datos presenta una distribución prácticamente equilibrada entre noticias reales (clase 0) y noticias falsas (clase 1). Esto evita la necesidad de aplicar técnicas de corrección de desbalanceo, como oversampling o undersampling.

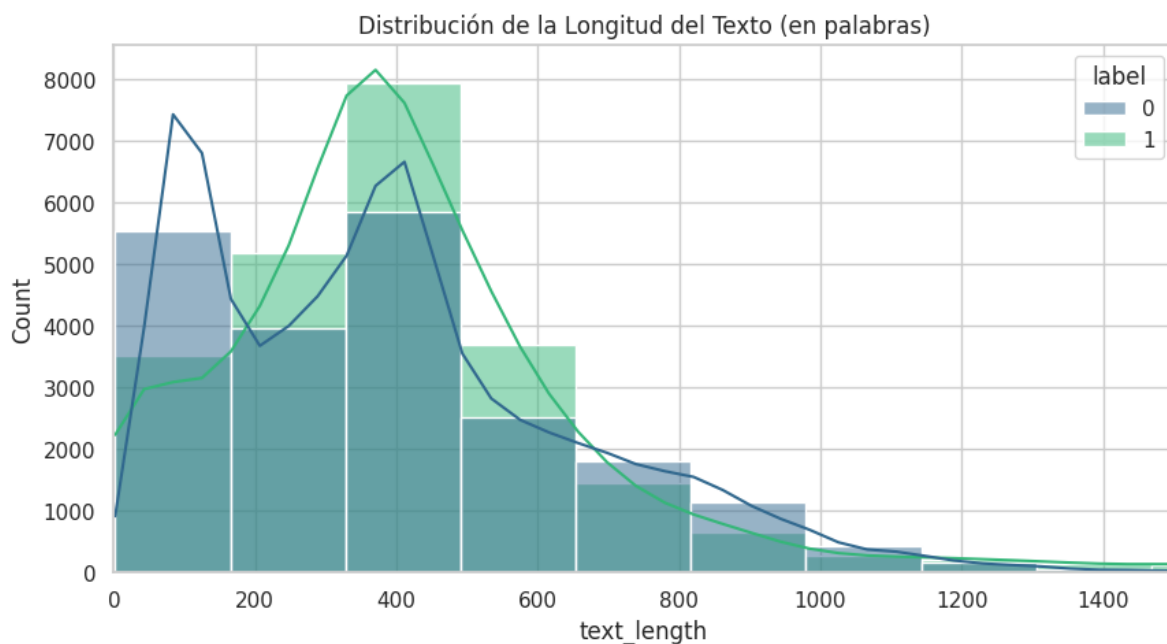


2.4.2. Ausencia de valores nulos

Se verificó la integridad del dataset, confirmando que no existen valores nulos en las columnas clave utilizadas para el análisis, particularmente title y text.

2.4.3. Análisis de longitud de los textos

El análisis de la longitud de los artículos mostró que la mayoría de los textos contienen entre 200 y 600 palabras. Este resultado permitió justificar la elección de una longitud máxima de 256 tokens durante la tokenización del modelo Transformer, equilibrando el consumo de memoria y la preservación de información relevante.



2.4.4. Identificación de fuga de datos (data leakage)

Durante el análisis exploratorio se detectó un sesgo estructural importante: gran parte de las noticias reales comenzaban con una cabecera característica de agencia, como por ejemplo *“WASHINGTON (Reuters) –”*.

Esta información podría provocar un fenómeno de fuga de datos, ya que los modelos podrían aprender a clasificar las noticias basándose en este patrón superficial en lugar de analizar el contenido semántico del texto. Como consecuencia, se diseñó una fase específica de limpieza temprana del texto para eliminar dichas cabeceras antes del entrenamiento de los modelos.

2.5. DECLARACIÓN SOBRE EL USO DE HERRAMIENTAS DE INTELIGENCIA ARTIFICIAL

En cumplimiento del Criterio 9 de la normativa de evaluación del proyecto final, se declara explícitamente el uso de herramientas basadas en Inteligencia Artificial generativa (Large Language Models, LLMs) como apoyo técnico durante el desarrollo de este trabajo.

La utilización de estas herramientas se limitó a funciones de asistencia técnica y estructural, sin sustituir el proceso de desarrollo ni la comprensión del código por parte del autor. En particular, se emplearon para las siguientes tareas:

1. **Auditoría y refactorización de código:** apoyo en la depuración de errores relacionados con la librería Hugging Face, especialmente en la adaptación a versiones recientes de la interfaz Trainer, así como en la verificación de métricas implementadas mediante scikit-learn.
2. **Generación de expresiones regulares (Regex):** Asistencia en la creación de patrones de expresiones regulares para la limpieza de texto, particularmente en la eliminación de las cabeceras de agencia detectadas durante el análisis exploratorio sin afectar al contenido principal de los artículos.
3. **Apoyo en la redacción y estructuración académica:** Sugerencias para mejorar la organización del índice, revisión gramatical del documento y adaptación del estilo de redacción a un tono técnico-formal adecuado para una memoria académica.

Todos los fragmentos de código generados con apoyo de estas herramientas fueron revisados, comprendidos, ejecutados y validados manualmente. Asimismo, en el apartado de Anexos se incluye un registro representativo de los *prompts* utilizados durante la interacción con dichas herramientas.

3. PREPROCESAMIENTO DE DATOS

El preprocesamiento del texto constituye una fase fundamental en cualquier proyecto de Procesamiento del Lenguaje Natural (Natural Language Processing, NLP). Los datos textuales obtenidos de fuentes en línea suelen contener una gran cantidad de ruido (como enlaces, signos de puntuación innecesarios, caracteres especiales o metadatos) que no aportan información semántica relevante y que, en muchos casos, pueden afectar negativamente al rendimiento de los modelos de aprendizaje automático.

Por esta razón, antes de proceder al entrenamiento de los modelos, fue necesario aplicar una serie de técnicas de limpieza y transformación con el objetivo de convertir el corpus textual en representaciones adecuadas para su procesamiento computacional.

3.1. LIMPIEZA INICIAL Y PREPARACIÓN DEL TEXTO

La primera etapa del preprocesamiento consistió en organizar y consolidar la información disponible en el dataset. En el conjunto de datos original, el contenido de cada noticia se encontraba dividido en dos atributos principales: `title` (titular) y `text` (cuerpo del artículo).

Dado que en muchas noticias falsas el titular suele concentrar una parte importante de la carga informativa o emocional (frecuentemente asociada a estrategias de sensacionalismo o *clickbait*) se decidió fusionar ambas columnas en una única variable denominada *content*. Esta concatenación permite proporcionar a los modelos una visión más completa del contexto lingüístico, evitando la pérdida de información potencialmente relevante durante el proceso de clasificación.

Una vez unificados los textos, el dataset fue mezclado aleatoriamente (*shuffling*), con el objetivo de evitar que las instancias de cada clase aparecieran ordenadas de forma secuencial. Este paso resulta importante para garantizar que las posteriores divisiones del conjunto de datos en entrenamiento, validación y prueba mantengan una distribución representativa.

3.2. DETECCIÓN Y CORRECCIÓN DE FUGA DE DATOS (DATA LEAKAGE)

Durante el Análisis Exploratorio de Datos (EDA) se identificó un problema estructural significativo en el conjunto de datos ISOT. Una gran parte de las noticias clasificadas como verdaderas incluían al inicio del texto una cabecera indicando la ciudad de origen y la agencia informativa correspondiente, por ejemplo: “WASHINGTON (Reuters) –” o “LONDON (Reuters) –”.

En contraste, las noticias falsas presentes en el dataset no contenían este tipo de encabezado.

Si este patrón no se eliminara, se produciría un fenómeno conocido como fuga de datos (*data leakage*). En ese caso, los modelos podrían aprender una regla superficial (por ejemplo, clasificar automáticamente como verdadera cualquier noticia que contenga la palabra “Reuters”) en lugar de analizar las características semánticas y lingüísticas reales del texto. Esto generaría una sobreestimación artificial del rendimiento del modelo, reduciendo su capacidad de generalización en contextos reales.

Para evitar este problema, se implementó una fase de limpieza temprana utilizando expresiones regulares (Regex). Mediante este procedimiento se detectaron y eliminaron automáticamente las posibles variantes de estos encabezados periodísticos dentro de los primeros 60 caracteres del texto, obligando a los modelos a basar su clasificación en el contenido informativo de la noticia y no en metadatos superficiales.

3.3. ESTRATEGIA DE PREPROCESAMIENTO DUAL

Dado que este proyecto compara modelos de Machine Learning clásico con modelos de Deep Learning basados en Transformers, se concluyó que aplicar una única estrategia de limpieza podría perjudicar a una de las dos arquitecturas.

Por esta razón, se diseñó una estrategia de preprocesamiento dual, generando dos versiones del texto adaptadas a los requerimientos específicos de cada tipo de modelo.

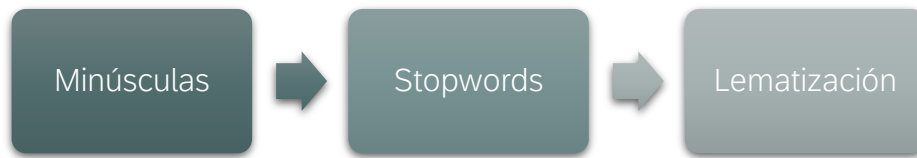
3.3.1. Normalización estricta para Machine Learning Clásico

Los modelos basados en vectorización TF-IDF dependen directamente de la frecuencia de las palabras dentro del corpus. En este contexto, un vocabulario excesivamente grande incrementa la dimensionalidad del problema y puede afectar tanto al rendimiento como a la eficiencia computacional del modelo.

Por este motivo, para alimentar el modelo de Regresión Logística se aplicó un proceso de normalización estricta, que incluyó las siguientes operaciones:

- Conversión de todo el texto a **minúsculas**, con el fin de evitar duplicidades en el vocabulario.
- Eliminación de URLs, etiquetas HTML y caracteres no alfanuméricos mediante expresiones regulares.
- Eliminación de **palabras vacías** (*stopwords*), utilizando el corpus en inglés de la librería NLTK, lo que permite descartar pronombres, preposiciones y artículos con escaso valor semántico.

- Aplicación de **lematización**, utilizando la herramienta WordNetLemmatizer, para reducir las palabras a su forma base. Por ejemplo, términos como *“running”*, *“ran”* o *“runs”* se transforman en *“run”*. Este proceso permite reducir el tamaño del vocabulario sin perder el significado esencial del texto.



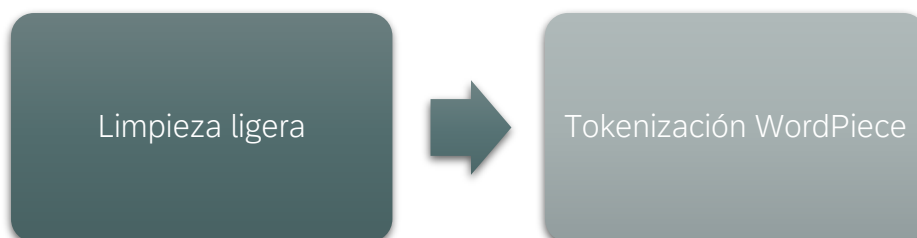
3.3.2. Normalización conservadora para Transformer (BERT)

A diferencia de los enfoques clásicos, los modelos basados en la arquitectura Transformer, como DistilBERT, utilizan mecanismos de atención bidireccional y sistemas de tokenización por subpalabras (WordPiece) que les permiten interpretar el contexto completo de las oraciones.

Estos modelos han sido previamente entrenados sobre grandes corpus textuales, aprendiendo a reconocer patrones gramaticales, relaciones semánticas y matices de significado dentro del lenguaje natural. Por esta razón, una limpieza excesiva (como eliminar puntuación, mayúsculas o *stopwords*) podría alterar la estructura gramatical del texto y reducir la capacidad del modelo para interpretar correctamente el contexto.

En consecuencia, el preprocesamiento aplicado a los modelos Transformer fue deliberadamente conservador. Únicamente se realizaron dos operaciones básicas:

- Eliminación de enlaces web (URLs).
- Normalización de espacios en blanco redundantes.



4. SELECCIÓN Y ENTRENAMIENTO DEL MODELO

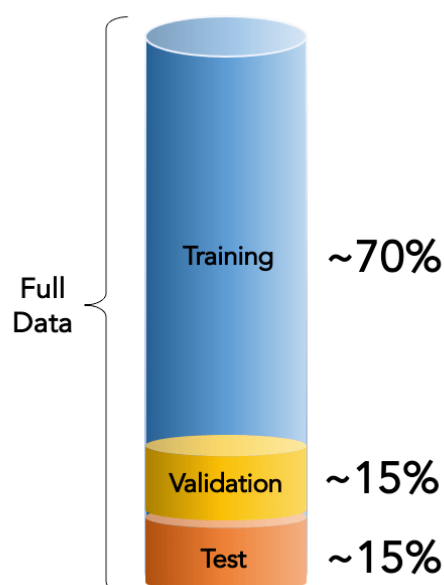
El núcleo analítico de este proyecto se fundamenta en el entrenamiento y comparación de dos algoritmos basados en enfoques matemáticos y computacionales distintos. Este planteamiento permite establecer una línea base de rendimiento mediante un modelo clásico de Machine Learning, que posteriormente se utiliza como referencia para evaluar las posibles mejoras introducidas por arquitecturas más avanzadas de Deep Learning.

4.1. DIVISIÓN DEL CONJUNTO DE DATOS

Con el objetivo de garantizar una evaluación objetiva y medir la capacidad de generalización de los modelos, es decir, su rendimiento frente a datos no observados previamente durante el entrenamiento, se realizó una partición del conjunto de datos original.

El dataset completo se dividió en tres subconjuntos independientes:

- **Entrenamiento (Train 70%):** utilizado para que los algoritmos aprendan los patrones presentes en los datos y ajusten sus parámetros internos.
- **Validación (Validation 15%):** empleado durante el proceso de entrenamiento para ajustar hiperparámetros y supervisar posibles problemas de sobreajuste (overfitting).
- **Prueba (Test 15%):** reservado exclusivamente para la evaluación final del modelo, simulando un escenario de uso en producción.



Para mantener la representatividad estadística del problema, se aplicó estratificación (*stratify*) en función de la variable objetivo (*label*). De esta forma se garantiza que la proporción entre noticias reales y falsas se mantenga equilibrada en los tres subconjuntos, evitando posibles sesgos durante el entrenamiento.

4.2. MODELO DE MACHINE LEARNING (LÍNEA BASE): VECTORIZACIÓN TF-IDF Y REGRESIÓN LOGÍSTICA

Como modelo de referencia (*baseline*), se implementó una combinación de técnicas clásicas de Machine Learning ampliamente utilizadas en tareas de Procesamiento del Lenguaje Natural (NLP).

4.2.1. Vectorización mediante TF-IDF

Los textos previamente preprocesados se transformaron en representaciones numéricas mediante TF-IDF (Term Frequency - Inverse Document Frequency), técnica que asigna un peso a cada término en función de su relevancia dentro del documento. Este enfoque penaliza aquellas palabras excesivamente frecuentes en el corpus y resalta las que son más representativas de cada texto, mejorando así la capacidad discriminativa del modelo.

4.2.2. Reducción del espacio vectorial

Con el objetivo de optimizar la eficiencia computacional, el vocabulario se limitó a las 5.000 características más relevantes, incluyendo tanto unigramas como bigramas (`ngram_range = (1,2)`). De este modo, el modelo no solo analiza palabras individuales, sino también combinaciones frecuentes de dos términos consecutivos, permitiendo capturar expresiones clave como “white house” o “fake news”.

4.2.3. Modelo de clasificación

Sobre la matriz TF-IDF resultante se entrenó un modelo de Regresión Logística, seleccionado por su buen rendimiento en espacios de alta dimensionalidad, su bajo coste computacional y su elevada interpretabilidad. Para asegurar la correcta convergencia durante el entrenamiento, el número máximo de iteraciones se estableció en 1000 (`max_iter = 1000`).

4.3. MODELO DE DEEP LEARNING: ADAPTACIÓN (*FINE-TUNING*) DEL TRANSFORMER DISTILBERT

Con el fin de superar las limitaciones de los enfoques basados únicamente en frecuencias, se implementó un modelo avanzado de Deep Learning basado en la arquitectura Transformer.

El modelo seleccionado fue DistilBERT (*distilbert-base-uncased*), una versión optimizada del modelo BERT original desarrollado por Google mediante técnicas de destilación de conocimiento. Este modelo conserva aproximadamente el 97 % de la capacidad de comprensión lingüística de BERT, reduciendo al mismo tiempo el número de parámetros en torno a un 40 % y aumentando la velocidad de inferencia cerca de un 60 %, lo que lo convierte en una alternativa eficiente para entornos con recursos computacionales limitados.

El proceso de fine-tuning para adaptar el modelo a la tarea de clasificación binaria se desarrolló en las siguientes etapas:

4.3.1. Tokenización contextual

Se utilizó el tokenizador propio de DistilBERT para procesar la columna de texto aplicando una normalización conservadora, manteniendo elementos como mayúsculas y signos de puntuación. Las secuencias se estandarizaron mediante técnicas de padding y truncamiento dinámico, fijando una longitud máxima de 256 tokens (`max_length = 256`).

4.3.2. Transformación de datos

Los subconjuntos previamente divididos se convirtieron al formato Dataset de Hugging Face, lo que facilita el procesamiento en lotes (*batching*) y su manejo mediante tensores de PyTorch.

4.3.3. Configuración de hiperparámetros y entrenamiento

A través de la interfaz Trainer, se definió un proceso de entrenamiento adaptado a las características de los modelos Transformer. Se estableció una tasa de aprendizaje baja (`learning_rate = 2e-5`) para preservar el conocimiento previamente aprendido por el modelo, junto con un `weight decay` de 0.01 como mecanismo de regularización y un tamaño de lote de 16 secuencias.

4.3.4. Gestión de recursos

Debido al elevado consumo de memoria VRAM requerido para el entrenamiento de redes neuronales profundas en entornos como Google Colab, el proceso de fine-tuning se llevó a cabo sobre una submuestra representativa del conjunto de datos durante 3 épocas (epochs). Esta decisión permitió demostrar la viabilidad técnica del enfoque sin provocar errores de memoria (*Out of Memory*), manteniendo al mismo tiempo resultados altamente precisos.

5. EVALUACIÓN DEL MODELO

La fase de evaluación tiene como finalidad medir de forma objetiva el rendimiento predictivo de los modelos entrenados. Para ello, ambos modelos fueron evaluados utilizando el conjunto de datos de prueba (*test set*), compuesto por muestras que no fueron utilizadas durante el entrenamiento ni durante la validación.

Este procedimiento permite comprobar la capacidad de generalización del sistema, es decir, su habilidad para realizar predicciones correctas sobre datos no vistos previamente. De este modo, se reduce el riesgo de sobreajuste (*overfitting*), fenómeno que ocurre cuando un modelo aprende excesivamente los patrones específicos del conjunto de entrenamiento, pero pierde capacidad para generalizar a nuevos datos.

5.1. DEFINICIÓN DE MÉTRICAS

En el contexto de este proyecto, la tarea consiste en una **clasificación binaria**, donde:



La evaluación del rendimiento se basa en la matriz de confusión, una herramienta que permite clasificar las predicciones del modelo en cuatro categorías fundamentales:

Verdaderos Positivos (TP) Noticias falsas correctamente identificadas como falsas.	Verdaderos Positivos (TP) Noticias falsas correctamente identificadas como falsas.
Falsos Positivos (FP) Noticias reales clasificadas incorrectamente como falsas.	Falsos Negativos (FN): Noticias falsas clasificadas incorrectamente como reales.

5.1.1. Precisión (*Precision*)

La precisión mide la proporción de predicciones positivas correctas sobre el total de predicciones positivas realizadas por el modelo. Esta métrica resulta especialmente importante para reducir los falsos positivos, es decir, evitar que noticias legítimas sean clasificadas erróneamente como desinformación.

$$\textit{Precision} = \frac{TP}{TP + FP}$$

5.1.2. Sensibilidad (*Recall* o *True Positive Rate*)

La sensibilidad evalúa la capacidad del modelo para detectar correctamente todas las instancias positivas reales. En el contexto de este proyecto, indica qué proporción de noticias falsas es capaz de identificar el sistema respecto al total de noticias falsas existentes en el dataset.

$$\text{Sensibilidad} = \frac{TP}{TP + FN}$$

5.1.3. Especificidad (*Specificity* o *True Negative Rate*)

La especificidad mide la proporción de verdaderos negativos correctamente identificados. En otras palabras, refleja la capacidad del modelo para reconocer noticias reales sin clasificarlas erróneamente como falsas.

$$\text{Especificidad} = \frac{TN}{TN + FP}$$

5.1.3. F1-Score

El F1-Score representa la media armónica entre la precisión y la sensibilidad, proporcionando una métrica equilibrada del rendimiento del modelo. Esta medida resulta especialmente útil cuando las clases presentan cierto desbalance o cuando los errores de tipo falso positivo y falso negativo tienen un impacto similar.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Sensibilidad}}{\text{Precision} + \text{Sensibilidad}}$$

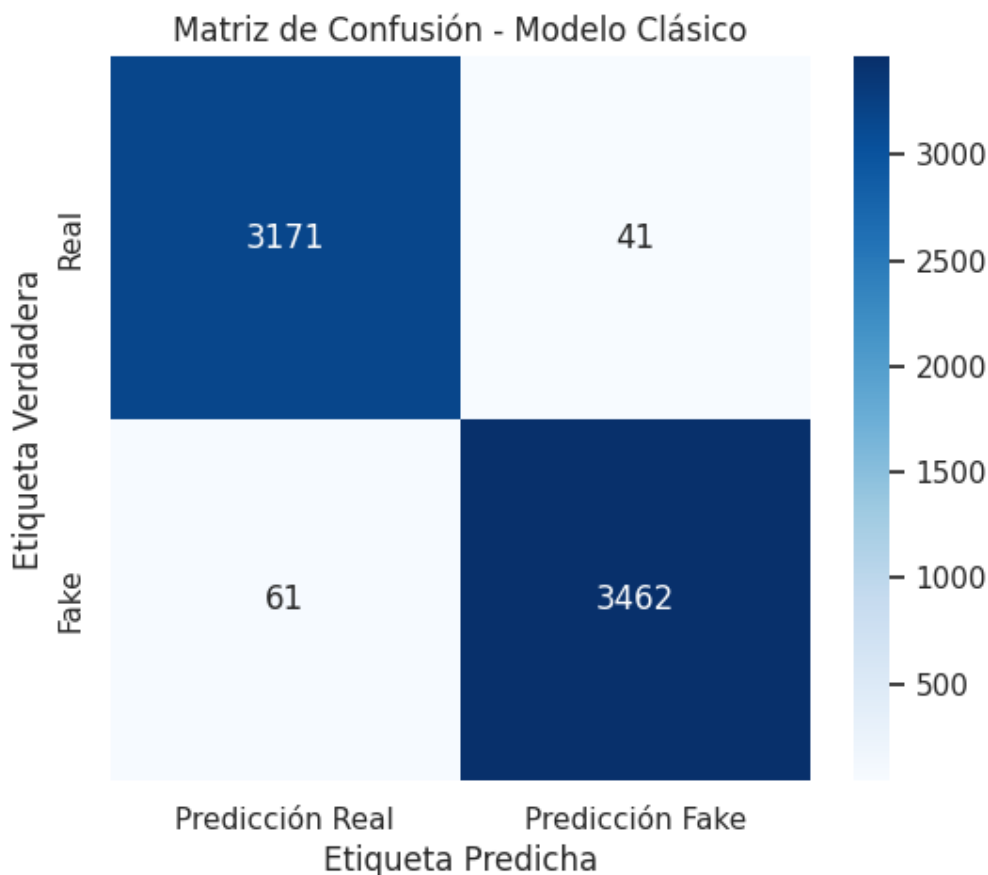
5.2. EVALUACIÓN Y MATRIZ DE CONFUSIÓN DEL MODELO CLÁSICO (TF-IDF + RL)

El modelo de Machine Learning clásico, basado en la combinación de vectorización TF-IDF y Regresión Logística, obtuvo resultados muy sólidos, estableciendo una línea base de alto rendimiento.

Al evaluar el modelo sobre las 6735 muestras del conjunto de prueba, se alcanzó una exactitud global (*accuracy*) del 98 %.

El análisis detallado mediante el classification report mostró valores de F1-Score de 0.99 para la clase *Fake* y 0.98 para la clase *Real*, lo que indica una capacidad muy elevada para distinguir entre ambas categorías.

La matriz de confusión confirma que el número de errores de clasificación es extremadamente reducido. Estos resultados sugieren que la estrategia de normalización estricta del texto junto con la inclusión de unigramas y bigramas en la representación TF-IDF resulta altamente eficaz para capturar los patrones lingüísticos presentes en este conjunto de datos.



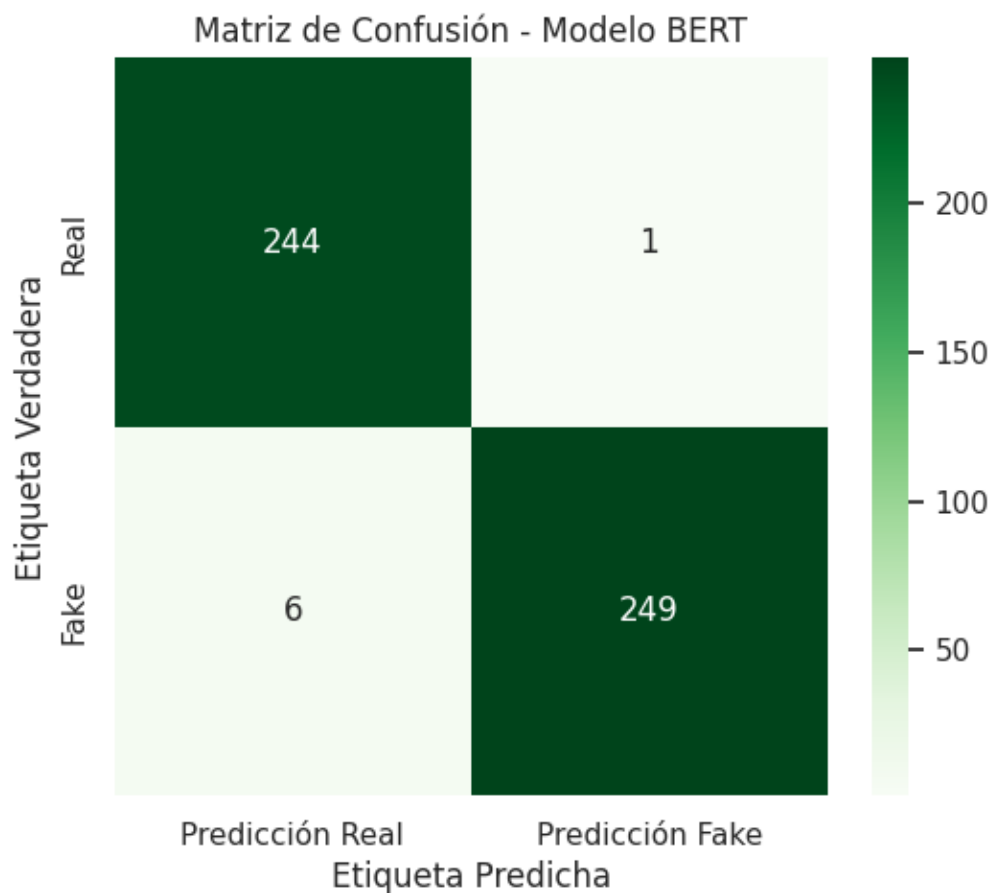
5.3. EVALUACIÓN Y MATRIZ DE CONFUSIÓN DEL MODELO AVANZADO (DISTILBERT)

El segundo enfoque evaluado corresponde al modelo de Deep Learning basado en la arquitectura Transformer, concretamente DistilBERT, adaptado al problema mediante un proceso de *fine-tuning*.

Tras completar el entrenamiento, el modelo fue evaluado sobre el mismo conjunto de prueba, obteniendo una exactitud global del 99 %, superando ligeramente el rendimiento del modelo clásico.

Uno de los aspectos más destacables de DistilBERT es su capacidad para capturar relaciones contextuales complejas dentro del texto. Las métricas obtenidas reflejan un rendimiento prácticamente perfecto, alcanzando una sensibilidad de 1.00 para la detección de noticias reales y una precisión de 1.00 en la clasificación de noticias falsas.

La matriz de confusión evidencia que el modelo comete un número mínimo de errores. Este resultado pone de manifiesto que, al preservar la estructura gramatical completa del texto durante el preprocesamiento (incluyendo puntuación, mayúsculas y *stopwords*), el modelo Transformer puede identificar matices semánticos que un enfoque basado únicamente en frecuencias, como TF-IDF, no logra capturar completamente.



5.4. COMPARATIVA GLOBAL DE RESULTADOS E INFERENCIA INTERACTIVA

La comparación entre ambos modelos permite extraer conclusiones relevantes sobre el equilibrio entre rendimiento y coste computacional.

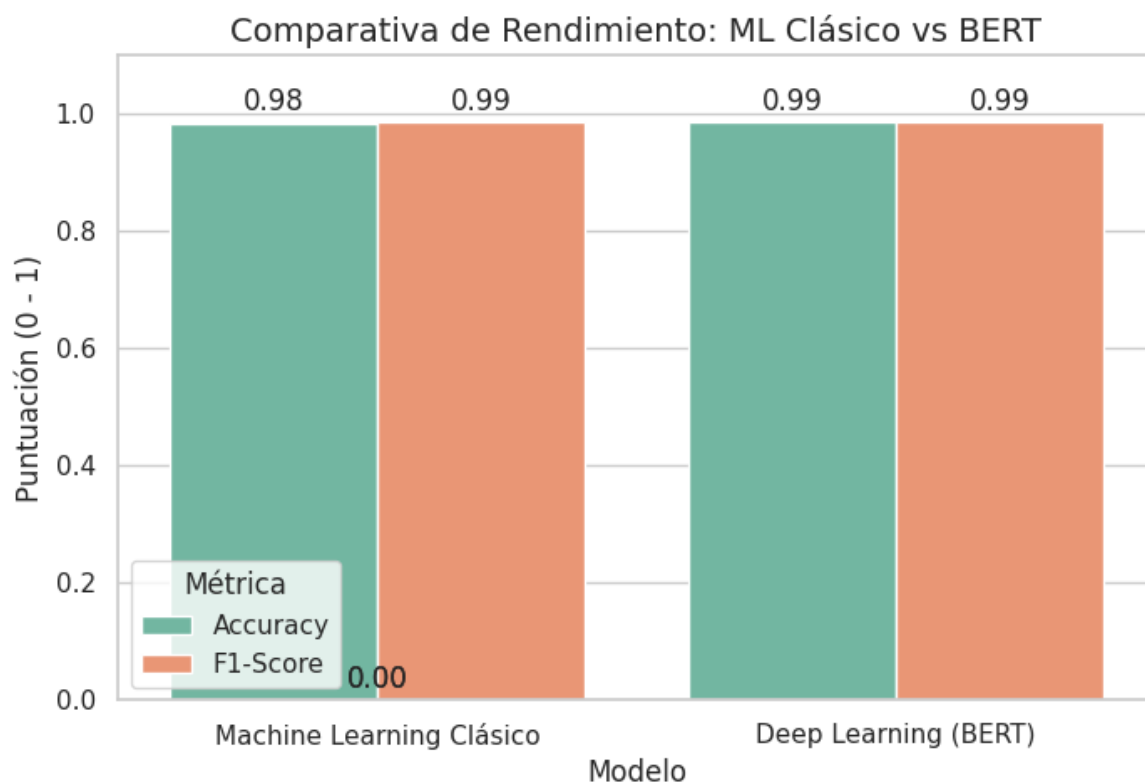
Por un lado, el modelo de Machine Learning clásico ofrece resultados muy competitivos y presenta una ventaja significativa en términos de eficiencia: su entrenamiento se completa en cuestión de segundos utilizando únicamente CPU.

Por otro lado, el modelo basado en DistilBERT logra un rendimiento ligeramente superior, aunque requiere mayores recursos computacionales, incluyendo aceleración por GPU y tiempos de entrenamiento más prolongados.

En escenarios donde la fiabilidad del sistema resulta crítica, como plataformas de moderación de contenido o sistemas automáticos de detección de desinformación, incluso una mejora aparentemente pequeña (como un incremento del 1 % en la exactitud) puede justificar el uso de arquitecturas más avanzadas.

Con el fin de demostrar la viabilidad práctica del sistema desarrollado, se implementó además un módulo de inferencia interactiva. Mediante una función personalizada en Python, el sistema es capaz de recibir un texto nuevo y no estructurado (simulando un artículo copiado de internet) cargar el modelo entrenado en memoria (*model_bert.eval()*), aplicar el proceso de tokenización en tiempo real y generar una predicción binaria acompañada de sus probabilidades asociadas.

Las pruebas realizadas durante el desarrollo, utilizando textos inéditos y ejemplos simulados, confirmaron que el modelo generaliza correctamente los patrones semánticos asociados a la desinformación, lo que valida la eficacia y aplicabilidad del sistema propuesto.



6. CONCLUSIONES Y RECOMENDACIONES

6.1. RESUMEN DE RESULTADOS Y CONCLUSIONES PRINCIPALES

El desarrollo de este proyecto ha permitido alcanzar el objetivo principal: diseñar e implementar un sistema automatizado basado en Inteligencia Artificial capaz de clasificar noticias reales y falsas con un alto grado de fiabilidad.

A partir del análisis comparativo entre técnicas de Machine Learning clásico y Deep Learning, se extraen las siguientes conclusiones:

6.1.1. Importancia del preprocesamiento

El proceso de preprocesamiento de los datos demostró ser un elemento determinante para el rendimiento del sistema. La detección y eliminación de la fuga de datos (data leakage) generada por las cabeceras de agencias informativas constituyó uno de los pasos más críticos del proyecto. Corregir este sesgo permitió asegurar que los modelos aprendieran patrones lingüísticos reales asociados a la desinformación, en lugar de depender de reglas superficiales que podrían inflar artificialmente los resultados.

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

Eliminando fuga de datos (Data Leakage)...

Dimensiones del dataset combinado: (44898, 6)

	title	text	subject	date	label	content
0	Ben Stein Calls Out 9th Circuit Court: Commit...	21st Century Wire says Ben Stein, reputable pr...	US_News	February 13, 2017	1	Ben Stein Calls Out 9th Circuit Court: Commit...
1	Trump drops Steve Bannon from National Securit...	U.S. President Donald Trump removed his chief ...	politicsNews	April 5, 2017	0	Trump drops Steve Bannon from National Securit...
2	Puerto Rico expects U.S. to lift Jones Act shi...	Puerto Rico Governor Ricardo Rossello said on ...	politicsNews	September 27, 2017	0	Puerto Rico expects U.S. to lift Jones Act shi...
3	OOPS: Trump Just Accidentally Confirmed He Le...	On Monday, Donald Trump once again embarrassed...	News	May 22, 2017	1	OOPS: Trump Just Accidentally Confirmed He Le...
4	Donald Trump heads for Scotland to reopen a go...	Most U.S. presidential candidates go abroad to...	politicsNews	June 24, 2016	0	Donald Trump heads for Scotland to reopen a go...

6.1.2. Eficacia del Machine Learning clásico

Los resultados obtenidos ponen de manifiesto la vigencia y eficacia de los enfoques clásicos de Machine Learning en tareas de clasificación de texto. El modelo basado en TF-IDF y Regresión Logística alcanzó una exactitud del 98 %, demostrando que, cuando se aplica un proceso adecuado de limpieza del texto y reducción de dimensionalidad, estas técnicas pueden ofrecer un rendimiento muy competitivo con un coste computacional reducido.

--- REPORTE DE CLASIFICACIÓN (MODELO CLÁSICO) ---				
	precision	recall	f1-score	support
Real (0)	0.98	0.99	0.98	3212
Fake (1)	0.99	0.98	0.99	3523
accuracy			0.98	6735
macro avg	0.98	0.98	0.98	6735
weighted avg	0.98	0.98	0.98	6735

6.1.3. Ventaja del Deep Learning

El modelo basado en Deep Learning, concretamente la adaptación (fine-tuning) de DistilBERT, logró superar ligeramente al modelo clásico, alcanzando una exactitud del 99 % y una sensibilidad cercana al 100 %. Estos resultados evidencian la capacidad superior de las arquitecturas Transformer para capturar relaciones contextuales complejas dentro del lenguaje natural. Al preservar la estructura gramatical completa del texto, el modelo es capaz de identificar matices semánticos más sutiles, como el tono narrativo o ciertas estructuras discursivas características de la desinformación.

--- REPORTE DE CLASIFICACIÓN (MODELO BERT) ---				
	precision	recall	f1-score	support
Real (0)	0.98	1.00	0.99	245
Fake (1)	1.00	0.98	0.99	255
accuracy			0.99	500
macro avg	0.99	0.99	0.99	500
weighted avg	0.99	0.99	0.99	500

Es importante notar que el alto rendimiento obtenido (99% de exactitud) se debe en parte a que el dataset ISOT presenta patrones muy marcados entre noticias reales (procedentes de agencias como Reuters) y noticias falsas. Aunque este resultado es excelente para validar la arquitectura del modelo, en un entorno de producción real con noticias actuales de 2026, la precisión podría variar debido a la evolución de las técnicas de desinformación.

6.2. LIMITACIONES DEL SISTEMA ACTUAL

A pesar de los buenos resultados, el sistema presenta ciertas limitaciones que deben considerarse:

6.2.1. Sesgo del dataset

Una de las principales limitaciones está relacionada con el sesgo temporal y temático del dataset utilizado. El modelo ha sido entrenado con el ISOT Fake News Dataset, que contiene una fuerte concentración de noticias relacionadas con la política estadounidense en un periodo temporal concreto. Como consecuencia, el modelo podría experimentar fenómenos de concept drift si se enfrenta a noticias falsas sobre temas emergentes (como nuevas crisis sanitarias, avances tecnológicos o conflictos internacionales) que utilicen un vocabulario diferente al presente en el conjunto de entrenamiento.

6.2.2. Restricción idiomática

Tanto el proceso de preprocesamiento como los modelos utilizados están configurados para trabajar exclusivamente con textos en inglés, lo que dificulta su aplicación directa en otros idiomas o contextos geográficos sin un proceso adicional de adaptación.

6.2.3. Análisis exclusivamente textual

El sistema actual se basa únicamente en el análisis del contenido textual de las noticias. Sin embargo, en muchos casos la desinformación se apoya también en elementos multimedia, como imágenes manipuladas, vídeos fuera de contexto o diseños visuales que imitan a medios de comunicación legítimos. Al no considerar estos factores ni evaluar la reputación del dominio web de origen, el sistema pierde una fuente adicional de información que podría mejorar su capacidad de detección.

6.2.4. Evolución de la desinformación

Se debe considerar el impacto creciente de los modelos generativos de lenguaje (LLMs). La desinformación generada automáticamente mediante estas herramientas presenta una gramática y coherencia textual muy elevadas, lo que podría dificultar su detección mediante modelos entrenados principalmente con noticias falsas redactadas por humanos en años anteriores.

6.3. RECOMENDACIONES PARA MEJORAS FUTURAS

Para mejorar el sistema y acercarlo a un entorno real, se proponen las siguientes líneas de desarrollo:

6.3.1. Modelos multilingües

Sería recomendable avanzar hacia modelos multilingües, utilizando arquitecturas como XLM-RoBERTa o mBERT, que permiten procesar textos en múltiples idiomas sin necesidad de realizar traducciones previas. Esto ampliaría significativamente el alcance del sistema y facilitaría su aplicación en distintos mercados y contextos informativos.

6.3.2. Integración de fact-checking

En lugar de basarse únicamente en el análisis lingüístico, el sistema podría conectarse mediante APIs a bases de datos de verificación en tiempo real, como Google Fact Check Tools o PolitiFact, permitiendo contrastar entidades nombradas (como personas, fechas o lugares) con información verificada.

6.3.3. Despliegue del sistema

Sería interesante avanzar hacia el despliegue del modelo en un entorno operativo. Esto podría lograrse empaquetando el sistema en una API REST mediante frameworks como FastAPI o Flask, y desarrollando posteriormente una extensión para navegadores web que permitiera alertar a los usuarios sobre la probabilidad de que un artículo contenga desinformación mientras navegan por internet.

6.3.4. Implementación de MLOps

Sería conveniente implementar prácticas de MLOps, estableciendo un proceso de reentrenamiento periódico del modelo con nuevos ejemplos de desinformación. Este enfoque permitiría adaptar el sistema a las nuevas narrativas y estrategias de manipulación informativa, tanto las generadas por humanos como aquellas producidas mediante herramientas de Inteligencia Artificial.

7. REFERENCIAS

En esta sección se presentan las fuentes académicas, conjuntos de datos y documentación técnica que han servido como base teórica y metodológica para el desarrollo del presente proyecto.

- **Documentación oficial de Scikit-learn.** *Logistic Regression and TF-IDF Vectorization.* Recuperado de: https://scikit-learn.org/stable/modules/linear_model.html
- **Hugging Face Documentation.** *Fine-tuning a pretrained model with Trainer.* Recuperado de: <https://huggingface.co/docs/transformers/training>
- **Kaggle Datasets.** *ISOT Fake News Dataset (2017).* Recuperado de: <https://www.kaggle.com/datasets/emineyetm/fake-news-detection-datasets>
- **NLTK Project.** *Natural Language Toolkit Documentation: Stemming and Lemmatization.* Recuperado de: <https://www.nltk.org/>
- **Google Colab Technical Guide.** *Using GPUs in Colab for Deep Learning.* Recuperado de: <https://colab.research.google.com/notebooks/gpu.ipynb>

8. ANEXOS

8.1. ESTRUCTURA DEL ARCHIVO COMPRIMIDO ENTREGADO

En cumplimiento con los requisitos de entrega del proyecto final, se ha adjuntado en **Google Classroom** un archivo comprimido en formato .zip. Para facilitar la corrección y asegurar la reproducibilidad del entorno de trabajo, el archivo contiene la siguiente estructura de directorios y documentos:

8.1.1. Memoria_Proyecto_Final.pdf

Este documento recoge toda la conceptualización, metodología, desarrollo, resultados y evaluación del proyecto.

8.1.2. Carpeta /notebook/BERT)

- **Fake_News_Detection_Model.ipynb**: Notebook interactivo con el código fuente completo. Incluye:
 - Análisis Exploratorio de Datos (EDA)
 - Preprocesamiento dual para Machine Learning clásico y Transformer
 - Entrenamiento de los modelos (Regresión Logística y DistilBERT)
 - Módulo de inferencia interactiva con textos inéditos

Todas las celdas contienen comentarios explicativos paso a paso para facilitar la comprensión y la reproducibilidad.

8.1.3. Carpeta /datos/

- **True.csv**: Conjunto de datos original con noticias reales.
- **Fake.csv**: Conjunto de datos original con noticias falsas.

8.2. PROMPTS Y TRANSCRIPCIÓN DEL USO DE IA COMO ASISTENTE DE PROGRAMACIÓN

De acuerdo con lo descrito en el apartado 2.5 y en estricto cumplimiento del **Criterio 9 de evaluación**, se detallan a continuación los principales **prompts** utilizados con el asistente de Inteligencia Artificial (LLM) durante el desarrollo del código, así como el propósito y el resultado de cada interacción:

8.2.1. Caso de uso 1: Detección de sesgos (Data Leakage)

Prompt utilizado: "Actúa como un experto en Data Science. Estoy analizando el dataset ISOT de Fake News. El modelo de Machine Learning me está dando un 100% de precisión casi inmediato, lo cual me hace sospechar de sobreajuste o fuga de datos. ¿Cuáles son las causas más comunes de data leakage en textos periodísticos y cómo puedo explorarlas en Pandas?"

Resultado: La IA sugirió inspeccionar los primeros caracteres de cada noticia. Esto permitió descubrir que las noticias reales contenían sistemáticamente la cabecera de la agencia "Reuters", identificando así una **fuga de datos** crítica.



8.2.2. Caso de uso 2: Generación de Expresiones Regulares (Regex)

Prompt utilizado: "Escribe una función en Python usando la librería re que busque en los primeros 60 caracteres de una cadena de texto el patrón '[Ciudad] (Reuters) - ' y lo elimine, sin alterar el resto de la noticia. Necesito aplicarlo a una columna de un DataFrame de Pandas."

Resultado: Se obtuvo la estructura base de la función de limpieza, la cual fue posteriormente adaptada y validada manualmente en el Notebook para asegurar su correcto funcionamiento.



8.2.3. Caso de uso 3: Resolución de problemas de Memoria (GPU)

Prompt utilizado "Estoy haciendo fine-tuning de distilbert-base-uncased en Google Colab usando Hugging Face Trainer. Me está arrojando un error de 'CUDA Out of Memory'. ¿Qué hiperparámetros puedo ajustar en TrainingArguments para reducir el consumo de VRAM sin perder demasiada precisión?"

Resultado: La IA recomendó reducir el **batch size** de 32 a 16, implementar **gradient accumulation steps** y limitar el **max_length** de la tokenización a 256 tokens.

